Digital Egypt Pioneers

وزارة الاتصـــــالات
وتكنولوجيا المعلومات

DIGITAL EGYPT PIONEERS INITIATIVE | AI & DATA SCIENCE TRACK | ROUND 5

Project Documentation

Real-Time Object Detection for Autonomous Vehicles



TEAM

01
Loai Ashraf Nour El-Deen Mohamed

02
Nour Nabil Ibrahim Mohammed

03
Mariam Mohamed Elsayed Tantawy

04
Karim Mohamed Alamir Sharaf

05
Logain Amr Ali Mahmoud

06
Mazen Reda Ahmed Murad

CONTENTS

SECTION 1
Project Planning & Management

SECTION 2
Literature Review

SECTION 3
Requirements Gathering

Contents

Click any entry to jump to it.

1. Project Planning & Management....3
1.1 Project Proposal....3
1.2 Project Plan....5
Timeline (Gantt chart)....5
Dependencies....6
Milestone 1: Data Collection, Exploration, and Preprocessing (Week 1)....6
Milestone 2: Object Detection Model Development (Weeks 2-3)....7
Milestone 3: ONNX Export, SageMaker Endpoint, and Real-Time Replay Testing (Week 4)....7
Milestone 4: MLOps and Monitoring (Weeks 4-5)....8
Milestone 5: Final Documentation and Presentation (Week 5)....8
Resource allocation....9
1.3 Task Assignment & Roles....9
Weekly assignments....9
1.4 Risk Assessment & Mitigation Plan....10
1.5 KPIs (Key Performance Indicators)....11

2. Literature Review....11
2.1 Related Work....11
2.2 Research Gap and Our Contribution....12
2.3 Feedback & Evaluation....12
2.4 Suggested Improvements....13
How the suggestions are addressed in this document....13
2.5 Final Grading Criteria....13

3. Requirements Gathering....13
3.1 Stakeholder Analysis....13
3.2 User Stories & Use Cases....14
User stories....14
Use cases....14
3.3 Functional Requirements....15
3.4 Non-functional Requirements....15

References....16

1. Project Planning & Management

1.1 Project Proposal

Overview. The project builds a real-time object detection model that detects and classifies pedestrians, cyclists, vehicles (cars and trucks), traffic signs, and obstacles in driving images and video, to support safety and decision-making in real-time driving scenarios. The detector is a lightweight YOLOv5s model with ECA and SE channel attention, following the design in [1], built in PyTorch. The project works only with existing public datasets and recorded driving video. It does not use vehicle hardware, sensors, or IoT services.

Cloud approach (AWS). As an AWS Machine Learning track project, the whole pipeline runs on AWS: versioned datasets in Amazon S3, SQL exploration with AWS Glue and Amazon Athena, training and fine-tuning on Amazon SageMaker, and the final ONNX model deployed on a SageMaker endpoint. A drift report built with Evidently triggers automated retraining through GitHub Actions or AWS Step Functions. Google Colab and Kaggle are used only for quick experiments and as a fallback.

Vertical slicing. To give every member end-to-end ownership, three members each own the full pipeline (data, fine-tuning, testing) for one group of object classes. The other three members own the shared YOLO architecture and training, the inference, export, and endpoint pipeline, and the MLOps and monitoring layer.

Objectives

- Build a YOLO-based detector for three class groups (pedestrians and cyclists, vehicles, traffic signs and obstacles) that works across different lighting conditions, road types, and environmental factors.
- Test whether the high accuracy reported on KITTI in [1] holds on BDD100K and in low light, motion blur, and occlusion. Measure the gap per class and per condition, and test whether augmentation and fine-tuning reduce it.
- Run the pipeline on AWS: store versioned datasets in Amazon S3, train and fine-tune on Amazon SageMaker, and deploy the final ONNX model on a SageMaker endpoint, instead of relying only on local or Colab environments.
- Show real-time suitability: measure FPS, latency, GFLOPs, and model size before and after ONNX export, and measure end-to-end latency while replaying recorded video to the endpoint.
- Validate the mapping between KITTI and BDD100K labels early in Week 1, so bounding box and class mismatches do not corrupt the merged dataset.
- Build an MLOps layer: Evidently drift reports on inference logs and an automated retraining trigger (GitHub Actions or AWS Step Functions) that starts a SageMaker training job, tested by simulating a data shift with existing data.

Scope

In scope	Out of scope
Detection of pedestrians and cyclists, vehicles (cars/trucks), traffic signs, and obstacles	Vehicle hardware, sensors, and live camera capture
Existing datasets: KITTI, BDD100K, COCO pre-trained weights (optional Open Images and Kaggle sets)	IoT services (for example AWS IoT Greengrass and IoT Core) and telemetry streaming from vehicles
AWS pipeline: Amazon S3, Glue and Athena, SageMaker training and endpoint, CloudWatch, Step Functions	Collecting new data on roads
Preprocessing (416/640 resize, normalization, augmentation) and early label-mapping validation	Object tracking, vehicle control, or driving decision logic

In scope	Out of scope
Transfer learning, per-slice fine-tuning, and attention variants (ECA, SE)	Safety certification of the system
Cross-dataset and per-condition evaluation (mAP, IoU)	
ONNX export, speed benchmarking, endpoint deployment, replay of recorded video, and a demo application	
Drift monitoring and an automated retraining trigger, simulated on existing data	
Final report, presentation, and project infographic	

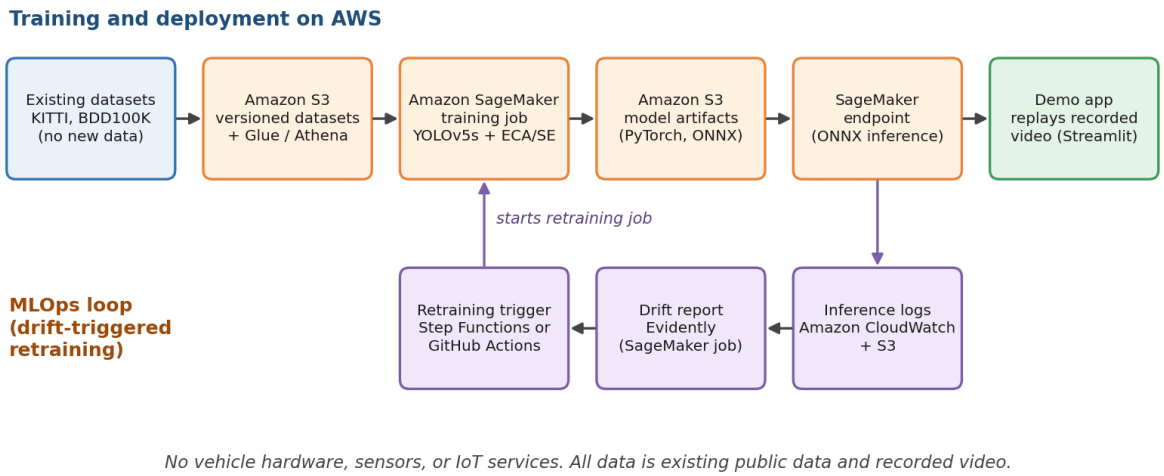


Figure 1. Proposed AWS pipeline: existing data in S3, training on SageMaker, the ONNX model on a SageMaker endpoint, and a drift-triggered retraining loop.

Tools and technologies. Python, PyTorch, Ultralytics YOLOv5 [6] with ECA [7] and SE [8] attention, OpenCV, ONNX and ONNX Runtime, DVC, MLflow, Evidently, Docker, Streamlit, Git and GitHub (with GitHub Actions). **AWS:** Amazon S3, AWS Glue, Amazon Athena, Amazon SageMaker (training jobs, processing jobs, endpoints), AWS Step Functions, Amazon CloudWatch, and AWS IAM.

Key focus areas

- **Real-time detection:** accurate detection and classification of objects at a speed suitable for driving, measured in FPS and latency.
- **Cloud integration on AWS:** S3 for data, SageMaker for training and deployment, and Step Functions or GitHub Actions for automation.
- **Transfer learning:** starting from pre-trained COCO weights for fast adaptation and better performance.
- **Generalization:** testing across datasets (KITTI to BDD100K) and across conditions (day, night, weather).
- **Vertical slicing and ownership:** each pipeline lead owns data, fine-tuning, and testing for their classes.
- **Continuous monitoring:** detecting drift and retraining automatically as new data becomes available, simulated with existing data.

- **Safety and reliability:** robust detection to protect passengers and pedestrians, reported honestly with its limits.

Expected outcome. A reproducible, documented detector trained and deployed on AWS, with measured accuracy and speed, a clear picture of how much accuracy is lost on unfamiliar data and conditions, and a working monitoring and automated retraining loop that adapts the model as data changes.

1.2 Project Plan

Time available and timeline decision. There are about 7 weeks between the first submission (16 Oct) and the final DEPI deadline (4 Dec 2026). One week is lost to midterm exams, leaving about 6 working weeks. The earlier 4-week plan was too short for the required work, and the milestones depend on each other. The plan therefore uses **5 weeks of work and keeps the 6th week as a safe-zone buffer** for fixes, integration, and final polish. IAM roles and SageMaker GPU quota are requested in Week 1, because they can take time.

DEPI documentation deadlines (from the project guidelines)

Item	DEPI deadline
Project Planning & Management	16 Oct 2026
Literature Review	16 Oct 2026
Requirements Gathering	16 Oct 2026
System Analysis & Design	6 Nov 2026
Implementation (Source Code & Execution)	30 Nov 2026
Final Presentation, Testing & Reports	4 Dec 2026

Timeline (Gantt chart)

Activity	Wk 1	Wk 2	Wk 3	Wk 4	Wk 5	Buffer
M1 Data collection, exploration, preprocessing						
AWS setup, S3 store, GPU quota request						
Label-mapping test, Athena SQL exploration						
M2 Model development and evaluation						
Baseline YOLOv5s on SageMaker, COCO transfer						
ECA/SE variants, cross-dataset evaluation						
M3 ONNX export, SageMaker endpoint, replay testing						
M4 MLOps, drift monitoring, automated retraining						
M5 Final report, presentation, infographic						
Safe-zone buffer: fixes, integration, polish						

Week numbers are working weeks. The midterm week is not counted, and its calendar position is fixed by the team once the exam schedule is known. Implementation must be finished by 30 Nov and reports by 4 Dec.

Dependencies

Dependency	Why it matters	How we reduce the risk
AWS setup before M1 and M2	S3, IAM roles, and SageMaker GPU quota are needed to store data and train	Request roles and quota in Week 1; use Colab for the first baseline if quota is late
M1 before M2	Training needs the preprocessed, split data, and a validated label mapping	Member 4 delivers the S3 data store and the tested mapping first; slice leads start with their own data in parallel
M2 before M3	Export and the endpoint need a trained model	Member 5 builds and tests the export and endpoint scripts on the pre-trained COCO baseline during Week 2-3
M3 before M4	Monitoring needs inference logs and confidence statistics	Member 6 builds the drift pipeline on logs from the baseline model and swaps in the final model later
M2, M3, M4 before M5	The final report needs all results	Members write report sections as each milestone ends, not at the end

Milestones, tasks, and deliverables

Milestone 1: Data Collection, Exploration, and Preprocessing (Week 1)

Objective: Collect, validate, explore, and prepare a unified, high-quality dataset on AWS for training the detector.

Area	Tasks	Owner
Data collection	Obtain, filter, and check annotations for datasets of their slice from KITTI and BDD100K (optional Open Images and Kaggle sets). Pedestrians and cyclists, vehicles (cars/trucks), and traffic signs and obstacles, one slice per member.	Members 1, 2, 3
AWS data store	Create the Amazon S3 bucket (raw, processed, labels) with versioning and IAM roles for the team, and use it as the DVC remote. Request SageMaker GPU instance quota and set a budget alarm.	Member 4
Label-mapping validation	Build the KITTI and BDD100K to shared-class mapping table and test it in Week 1, before any merge: automated checks for class names, box coordinates inside the image, empty or duplicate boxes, and overlays on sample images. Fix every mismatch before training.	Member 4, with Members 1, 2, 3 checking their slice
Data exploration	Catalog the annotation tables with AWS Glue and use Amazon Athena SQL to analyze the spatial representation of bounding boxes: locations, sizes, aspect ratios; visualize samples with their boxes.	Member 5
	Analyze class distribution, class imbalance, image resolution, label accuracy, and lighting, weather, and road-type coverage. Finalize the Dataset Exploration Report.	Member 6
Preprocessing	Resize images (640x640 as in [1], with 416x416 tested for speed) and augment: random cropping, flipping, rotation, plus brightness and blur changes for low light and motion blur. Keep boxes consistent.	Members 1, 2, 3
	Normalize pixel values, convert labels to YOLO format, create sequence-aware train/validation/test splits so near-identical video frames do not cross splits, and store the processed data in S3.	Member 4

Deliverables: Dataset Exploration Report (class and box statistics, imbalance, quality, conditions, and label-mapping test results); Preprocessed Data (clean, augmented, normalized dataset with boxes, versioned in S3 and DVC).

Milestone 2: Object Detection Model Development (Weeks 2-3)

Objective: Develop, train, and evaluate the YOLO detector on Amazon SageMaker, including attention variants and cross-dataset tests.

Area	Tasks	Owner
Model selection	Implement the base YOLOv5s detector in PyTorch; add ECA and SE attention variants from [1]; justify YOLO over SSD and Faster R-CNN by the speed and accuracy balance. YOLOv11n is an optional extra baseline.	Member 4
Model training	Apply transfer learning from pre-trained COCO weights and run the base training as an Amazon SageMaker training job that reads data from S3 and writes model artifacts to S3; track all runs in MLflow.	Member 4
	Fine-tune the baseline for their object classes and the environmental conditions in their data, using SageMaker training jobs.	Members 1, 2, 3
Model evaluation	Evaluate each slice with mAP@50, mAP@50-95, and IoU. Test on the training dataset (KITTI) and on BDD100K, and report results per class and per condition (day/night, weather).	Members 1, 2, 3
	Compile the overall metrics across all classes into the Model Evaluation Report; set up experiment tracking and data versioning.	Member 6
Model optimization	Measure FPS, latency, GFLOPs, and model size; compare 640 and 416 input sizes and FP16; export to ONNX and prepare the inference script for the endpoint.	Member 5

Deliverables: Model Evaluation Report (mAP, IoU, FPS, per-class and per-condition results, cross-dataset gap); Final Model (trained PyTorch weights and ONNX export, stored in S3).

Milestone 3: ONNX Export, SageMaker Endpoint, and Real-Time Replay Testing (Week 4)

Objective: Deploy the model on AWS, show that it runs in real time, and test it on driving video, using recorded data only.

Area	Tasks	Owner
Cloud deployment	Deploy the final ONNX model on an Amazon SageMaker real-time endpoint with an inference script. Keep the endpoint running only while testing to control cost.	Member 5
Benchmark	Run the ONNX model locally with ONNX Runtime (laptop GPU and CPU-only runs) and through the endpoint; record FPS and latency; compare accuracy before and after export.	Member 5
Real-time replay	Build an OpenCV pipeline that replays recorded video frame by frame to the endpoint as a simulated camera stream, a Streamlit demo application, and CloudWatch metrics for latency and errors.	Member 5
Real-time testing	Evaluate their slices on existing video sequences (BDD100K, KITTI) across urban streets, highways, and foggy, rainy, and night conditions.	Members 1, 2, 3
Model refinement	Fine-tune the combined model to correct the accuracy drops found during testing.	Member 4
Documentation	Document all environmental challenges and the adjustments made, and write the Testing Report.	Member 6

Deliverables: Deployed Model (ONNX model on a SageMaker endpoint with a demo application that runs on recorded video); Testing Report (results per condition, endpoint latency, problems found, adjustments).

Milestone 4: MLOps and Monitoring (Weeks 4-5)

Objective: Monitor the deployed model and automate retraining, tested by simulating a data shift with existing data.

Area	Tasks	Owner
MLOps setup	Build the DVC pipeline and MLflow tracking and model registry; add GitHub Actions checks. Version every dataset, run, and model.	Member 6
Drift monitoring	Run Evidently drift reports (as a SageMaker processing job) on inference logs and confidence statistics stored in S3. Simulate the shift (for example KITTI daytime data to BDD100K night and weather subsets) and set the drift threshold.	Member 6
Automated triggering	When drift exceeds the Evidently threshold, trigger retraining automatically, simulated with a GitHub Actions workflow or an AWS Step Functions state machine that starts a SageMaker training job, evaluates the new model, and registers and deploys it only if it passes. Log low-confidence frames for review.	Member 5, with Member 6

Deliverables: MLOps Report (pipeline, monitoring, retraining strategy); Monitoring Setup (drift reports, thresholds, alerts, trigger workflow, documentation).

Milestone 5: Final Documentation and Presentation (Week 5)

Objective: Document the project and prepare the final presentation and infographic.

Area	Tasks	Owner
Final report	Summarize the project from data collection to results, and its societal and engineering impact.	Member 4
	Document edge-case challenges, solutions, and possible future improvements (for example more object classes) for their slice.	Members 1, 2, 3
Final presentation	Present the architecture, real-world applications, AWS pipeline, results, and a live demo; show each member's responsibility and completed work. Prepare the single-slide project infographic.	Member 4 with the team

Deliverables: Final Project Report; Final Presentation; Project Infographic; all documents and code uploaded to GitHub.

Milestone summary

Milestone	Weeks	Key deliverables
1. Data Collection, Exploration & Preprocessing	1	Dataset Exploration Report, Preprocessed Data
2. Object Detection Model Development	2-3	Model Evaluation Report, Final Model
3. ONNX Export, SageMaker Endpoint & Replay Testing	4	Deployed Model, Testing Report
4. MLOps & Monitoring	4-5	MLOps Report, Monitoring Setup
5. Final Documentation & Presentation	5	Final Project Report, Final Presentation, Infographic

Milestone	Weeks	Key deliverables
Safe-zone buffer	6	Fixes, integration, final polish

Resource allocation

Resource	Allocation
People	6 team members: 3 vertical pipeline leads (Members 1-3) and 3 platform leads (Members 4-6), see 1.3
Data	KITTI, BDD100K, COCO pre-trained weights; optional Open Images and Kaggle traffic-sign sets. Existing data only, stored in Amazon S3.
AWS services	Amazon S3, AWS Glue, Amazon Athena, Amazon SageMaker (training, processing, endpoints), AWS Step Functions, Amazon CloudWatch, AWS IAM. AWS access is provided to the team; budget alarm set in Week 1.
Compute	SageMaker training instances (GPU quota requested in Week 1); Google Colab and Kaggle for quick experiments and as a fallback; laptop GPU and CPU-only runs for local speed tests
Software	Python, PyTorch, YOLOv5, OpenCV, ONNX Runtime, DVC, MLflow, Evidently, Docker, Streamlit
Collaboration	Git and GitHub with a branching workflow, GitHub Actions for checks and triggers; all documents uploaded to GitHub as required

1.3 Task Assignment & Roles

Member	Name	Role
Team Member 1	Loai Ashraf Nour El-Deen Mohamed	Pedestrian & Cyclist Pipeline Lead (end-to-end data, fine-tuning, and testing for human objects)
Team Member 2	Nour Nabil Ibrahim Mohammed	Vehicle Pipeline Lead (end-to-end data, fine-tuning, and testing for cars/trucks)
Team Member 3	Mariam Mohamed Elsayed Tantawy	Traffic Sign & Obstacle Pipeline Lead (end-to-end data, fine-tuning, and testing for road signs/hazards)
Team Member 4	Karim Mohamed Alamir Sharaf	Core YOLO Architecture, S3 Data Store & Base Training Infrastructure
Team Member 5	Logain Amr Ali Mahmoud	Inference, Export, SageMaker Endpoint & Automated Retraining Trigger
Team Member 6	Mazen Reda Ahmed Murad	MLOps, Drift Monitoring & Continuous Retraining

Weekly assignments

Member	Week 1	Week 2	Week 3	Week 4	Week 5
Member 1 (Pedestrian & Cyclist)	Collect, filter, augment pedestrian and cyclist data; check label mapping	Fine-tune YOLO on the slice (SageMaker)	Evaluate mAP/IoU; cross-dataset and per-condition tests	Replay tests on slice; fix drops	Document edge cases and future improvements
Member 2 (Vehicle)	Collect, filter, augment vehicle data; check label mapping	Fine-tune YOLO on the slice (SageMaker)	Evaluate mAP/IoU; cross-dataset and per-condition tests	Replay tests on slice; fix drops	Document edge cases and future improvements

Member	Week 1	Week 2	Week 3	Week 4	Week 5
Member 3 (Sign & Obstacle)	Collect, filter, augment traffic sign and obstacle data; check label mapping	Fine-tune YOLO on the slice (SageMaker)	Evaluate mAP/IoU; cross-dataset and per-condition tests	Replay tests on slice; fix drops	Document edge cases and future improvements
Member 4 (Core YOLO, data store)	AWS setup, S3 + DVC store, label-mapping test, normalization, splits	Base YOLOv5s and COCO transfer on SageMaker	ECA/SE variants; merge slice models	Refine combined model after tests	Final report and presentation
Member 5 (Inference, endpoint)	Glue catalog and Athena analysis of bounding boxes	Speed baseline (FPS, GFLOPs, size)	FP16, 416 vs 640, ONNX export, inference script	SageMaker endpoint, replay pipeline, demo app	Automated retraining trigger (Step Functions / GitHub Actions)
Member 6 (MLOps)	Class imbalance, Exploration Report	MLflow and DVC set up	Model Evaluation Report	Testing Report; Evidently drift job	Drift threshold and alerts, MLOps Report

1.4 Risk Assessment & Mitigation Plan

Likelihood (L) and impact (I) are rated Low, Medium, or High.

#	Risk	L	I	Mitigation
R1	Schedule too tight (midterm week, dependent milestones)	High	High	5-week plan with a 6th safe-zone buffer week; parallel preparation of export, endpoint, and monitoring scripts; weekly checkpoint
R2	SageMaker GPU quota or IAM setup delayed	Med	High	Request quota and IAM roles in Week 1; start with small instances; use Colab for the first baseline if needed
R3	AWS cost overrun (training jobs, endpoint left running)	Med	Med	Budget alarm in Week 1; keep the endpoint up only during tests; train on subsets first; stop idle resources
R4	Class imbalance across merged datasets (few cyclist or obstacle samples)	High	Med	Check class counts with Athena in Week 1; augment under-represented classes; re-balance splits
R5	Accuracy drops on BDD100K (night, weather, blur, occlusion)	High	High	Cross-dataset testing is the study itself; augmentation and fine-tuning on BDD100K; per-condition reporting
R6	KITTI and BDD100K label or box mismatches break the merged dataset	Med	High	Automated label-mapping tests and sample overlays in Week 1, before training; mapping table under version control
R7	BDD100K is large and slow to download	Med	Med	Use the image subset needed for detection; upload it to S3 once and share it
R8	Accuracy drops after ONNX or FP16 export	Med	Med	Compare mAP before and after export; keep the FP32 model if the drop is too large
R9	Slice models do not merge cleanly into one model	Med	High	Shared class list and label format from Week 1; Member 4 owns the merge from Week 3
R10	Drift cannot be measured without ground truth	Med	Med	Monitor input and confidence statistics; review low-confidence frames by hand; simulate shift with labelled subsets

#	Risk	L	I	Mitigation
R11	AWS credentials leaked in the GitHub repository	Low	High	Never commit keys; use IAM roles and GitHub Actions secrets; least-privilege permissions
R12	Results cannot be reproduced later	Low	High	Version data, code, and models with S3, DVC, MLflow, and Git; document commands in the README

1.5 KPIs (Key Performance Indicators)

Targets are proposed values for the team to confirm with the advisor.

KPI	How it is measured	Proposed target
Detection accuracy	mAP@50 and mAP@50-95 per slice and overall; IoU of predicted boxes	mAP@50 of at least 0.70 per slice
Real-time speed	FPS and per-frame latency of the ONNX model locally, before and after export	At least 30 FPS on a laptop GPU; CPU results reported
Endpoint performance	Median and 95th percentile end-to-end latency and error rate of the SageMaker endpoint during replay	Reported for the whole replay; no failed requests
Accuracy kept after export	mAP before versus after ONNX/FP16	Drop of at most 2 mAP points
Generalization gap	mAP on KITTI versus BDD100K, per class and per condition, before and after augmentation and fine-tuning	Gap measured for all classes; reduction reported
Label-mapping quality	Automated Week 1 tests on the merged KITTI and BDD100K data	Zero invalid boxes and zero unmapped classes before training
AWS integration	Datasets in S3, training on SageMaker, model on an endpoint	All three working end to end
Automated retraining	Simulated drift above the Evidently threshold starts the retraining workflow	Training job starts automatically; one-command rerun
Condition coverage	Slices evaluated in day, night, and weather subsets	100% of slice and condition combinations
Reproducibility	Another member can rerun the pipeline from the repository README	Full rerun without manual fixes
Delivery	Milestone deliverables finished within the 5 working weeks	5 of 5 milestones, buffer week unused or used for polish

2. Literature Review

2.1 Related Work

Ref	Work	Main idea	Relevance to our project
[1]	Adam and Tapamo (2025), Enhancing YOLOv5 for Autonomous Driving	Adds ECA and SE channel attention to YOLOv5s for efficient detection on edge devices. Trained on KITTI; reports 98.4% mAP@50 for a vehicle-only model and about 13 GFLOPs, 9.1 MB, and 27 ms for its smallest variant. Reports accuracy drops on BDD100K in low light and motion blur.	Base paper. We rebuild the models, re-measure the results, and study the gap on BDD100K per class and condition.

Ref	Work	Main idea	Relevance to our project
[2]	Liang et al. (2022), Edge YOLO	Real-time object detection for autonomous vehicles using edge-cloud cooperation.	Shows the need for fast, lightweight detection and the cost of depending on connectivity; motivates measuring speed after export and through an endpoint.
[3]	Alimov and Meiramkhanov (2024), Domain Generalization in Autonomous Driving	Evaluates YOLOv8s, RT-DETR, and YOLO-NAS on ROAD-Almaty, a dataset from a new location, to study how detectors generalize.	Supports our cross-dataset protocol: strong results on one dataset do not guarantee results on another.
[4]	Geiger et al. (2013), KITTI	Benchmark dataset for autonomous driving perception; about 7,481 labelled training images for object detection.	Training and standard evaluation data.
[5]	Yu et al. (2020), BDD100K	Large driving dataset of 100,000 video clips with different places, weather, and times of day.	Harder test data with night, weather, and traffic sign classes.
[6]	Jocher (2020), Ultralytics YOLOv5	PyTorch implementation of the single-stage YOLOv5 detector, with COCO pre-trained weights.	Base detector and transfer learning source.
[7], [8]	Wang et al. (2019) ECA-Net; Hu et al. (2017) SE-Net	Channel attention modules that reweight feature channels at low extra cost.	Attention variants tested in the model.

2.2 Research Gap and Our Contribution

- Strong scores on KITTI describe standard driving conditions. The drop on BDD100K reported in [1] is not broken down by condition. We repeat the evaluation with sequence-aware splits and report results per class and per condition, and test whether augmentation and fine-tuning reduce the gap.
- Speed and size are reported on the authors' own setup. We re-measure FPS and latency locally and through a SageMaker endpoint, before and after export.
- The cited work stops at the trained model. We add a cloud pipeline with automated, drift-triggered retraining, simulated on existing data, to show how the model could be kept up to date.

Feedback and suggestions received from the lecturer are recorded below.

Lecturer name	Date
Eng. Randa	16 Oct 2026

2.3 Feedback & Evaluation

The project proposal demonstrates a solid, well-structured approach to real-time object detection using optimized YOLO architectures with attention mechanisms (ECA/SE). The vertical slicing strategy and clear milestone distribution across 5 weeks with a buffer week are highly commendable. The integration of MLOps components (DVC, MLflow, Evidently) elevates the engineering maturity of the graduation project. However, given the AWS Machine Learning track requirements, the current reliance solely on local or Colab/Kaggle environments needs to be bridged with native cloud services.

2.4 Suggested Improvements

1. Cloud Integration: Since this is an AWS Machine Learning track project, it is strongly recommended to integrate core AWS services into the pipeline, such as storing versioned datasets in Amazon S3, utilizing Amazon SageMaker for model training/fine-tuning, and deploying the final ONNX model using SageMaker Endpoints rather than relying solely on local/Colab environments.

2. Data Pipeline Scalability: Ensure that the data mapping between KITTI and BDD100K is rigorously tested early in Week 1 to avoid bounding box label mismatches during the multi-dataset merge.

3. Automated Triggering: Enhance the MLOps monitoring loop to simulate automated retraining triggers using GitHub Actions or AWS Step Functions when drift exceeds the Evidently threshold.

How the suggestions are addressed in this document

Suggestion	How we address it	Where
Cloud Integration	Versioned datasets in Amazon S3 (also the DVC remote); training and fine-tuning as SageMaker training jobs; the final ONNX model deployed on a SageMaker endpoint. Colab is only a fallback.	Milestones 1-3, Figure 1, Section 1.2
Data Pipeline Scalability	KITTI and BDD100K label mapping built and tested automatically in Week 1, with overlays on sample images, before any merge or training.	Milestone 1, risk R6, KPIs
Automated Triggering	Evidently drift above the threshold starts retraining automatically through a GitHub Actions workflow or an AWS Step Functions state machine that starts a SageMaker training job.	Milestone 4, KPIs

2.5 Final Grading Criteria

Criterion	Covers	Max marks	Marks awarded
Documentation	Planning, requirements, analysis and design documents		
Implementation	Source code, version control, deployment and execution		
Testing	Test plan, test results, quality assurance		
Presentation	Final presentation, reports, demonstration		
Total			

3. Requirements Gathering

3.1 Stakeholder Analysis

Stakeholder	Needs and expectations	Influence
Road users (vehicle occupants, pedestrians, cyclists)	Indirect beneficiaries: reliable detection of road users and signs, including at night and in bad weather	Medium
Autonomous vehicle industry and researchers	A lightweight, fast, honestly evaluated detector, and clear evidence of how it behaves on new data	High

Stakeholder	Needs and expectations	Influence
Development team (6 members)	Clear ownership of each pipeline, shared data and class list, reproducible training and evaluation on AWS	High
MLOps and maintenance engineers	Experiment tracking, drift alerts, and a retraining pipeline that starts automatically	Medium
Advisor and DEPI evaluators (AWS Machine Learning track)	Deliverables on time, clear division of work, real use of AWS services, a working demonstration, and results reported honestly	High

3.2 User Stories & Use Cases

User stories

- As a **researcher**, I want the model evaluated on a second dataset so that I know whether the reported accuracy holds on unfamiliar data.
- As a **pipeline lead**, I want to prepare, fine-tune, and test only my object classes end to end so that I own the results for my slice.
- As a **data engineer on the team**, I want the KITTI and BDD100K labels mapped and tested in Week 1 so that the merged dataset has no mismatched boxes.
- As a **data analyst on the team**, I want to query bounding box locations and class counts with SQL in Athena so that I can find imbalances early.
- As a **cloud engineer on the team**, I want datasets in S3, training on SageMaker, and the model on an endpoint so that the project runs on AWS end to end.
- As an **evaluator**, I want to run the detector on a recorded driving video in a demo so that I can see real-time detection working.
- As an **MLOps engineer**, I want drift above a threshold to start retraining automatically so that the model is updated before accuracy falls.

Use cases

ID	Use case	Actor	Main flow
UC1	Detect objects in a frame or video	Evaluator, team member	A frame or recorded video is sent to the endpoint or the local model; objects are detected and classified; boxes, classes, and confidence scores are shown
UC2	Replay driving video in real time	Inference engineer	Recorded video is read frame by frame with OpenCV and sent to the SageMaker endpoint; FPS and latency are logged to CloudWatch
UC3	Explore the dataset	Pipeline leads, Members 5 and 6	Annotations in S3 are cataloged by Glue; Athena SQL queries return bounding box locations, sizes, and class distributions
UC4	Validate the label mapping	Member 4, pipeline leads	The mapping table is applied to KITTI and BDD100K; automated checks and sample overlays report mismatches; fixes are made before the merge
UC5	Train and evaluate on SageMaker	Pipeline leads, Member 4	A SageMaker training job reads data from S3, trains or fine-tunes the model, writes artifacts to S3; mAP and IoU are reported per class and per condition

ID	Use case	Actor	Main flow
UC6	Monitor drift and retrain automatically	MLOps engineer, Member 5	An Evidently drift job compares inference logs with a reference set; above the threshold, GitHub Actions or Step Functions starts a SageMaker training job; the new model is registered and deployed if it passes

3.3 Functional Requirements

ID	Requirement
FR-01	The system shall detect and classify pedestrians, cyclists, vehicles (cars/trucks), traffic signs, and obstacles in images and video frames.
FR-02	The system shall output a bounding box, class label, and confidence score for each detected object.
FR-03	The system shall process recorded video frame by frame as a simulated real-time camera stream.
FR-04	Datasets shall be stored in versioned Amazon S3 buckets, cataloged with AWS Glue, and queryable with Amazon Athena.
FR-05	The KITTI and BDD100K label mapping shall be validated by automated tests in Week 1 before any dataset merge.
FR-06	Images shall be resized (640x640, with 416x416 tested), normalized, and augmented (random cropping, flipping, rotation, brightness and blur) before training.
FR-07	Train, validation, and test splits shall be sequence-aware so near-identical video frames do not cross splits.
FR-08	The model shall be trained and fine-tuned with transfer learning from pre-trained COCO weights using Amazon SageMaker training jobs.
FR-09	The system shall report mAP and IoU per slice, per class, per condition, and across datasets (KITTI to BDD100K).
FR-10	The system shall export the model to ONNX and deploy it on an Amazon SageMaker endpoint.
FR-11	The system shall report FPS, latency, GFLOPs, and model size before and after export, and endpoint latency during replay.
FR-12	The system shall log low-confidence frames and inference statistics for review and monitoring.
FR-13	The system shall produce Evidently drift reports and raise an alert when a set threshold is crossed.
FR-14	The system shall start retraining automatically when drift exceeds the threshold, through GitHub Actions or AWS Step Functions starting a SageMaker training job.
FR-15	The system shall track experiments and register model versions (MLflow) and version data and code (DVC, Git).
FR-16	The system shall provide a demo application that runs the detector on recorded video.

3.4 Non-functional Requirements

Category	Requirement
Performance	Real-time inference (proposed at least 30 FPS for the local ONNX model on a laptop GPU) with low per-frame latency; endpoint latency measured and reported; accuracy kept within a small margin after export
Cost	Cloud cost stays within the available AWS budget: budget alarm, endpoint running only during tests, idle resources stopped

Category	Requirement
Security	Least-privilege IAM roles; no credentials in the repository (GitHub Actions secrets); only public datasets, used under their licenses; no personal data collected
Reproducibility	All data, code, runs, and models versioned; the pipeline can be rerun from the README without manual fixes
Robustness	Acceptable detection in urban streets, highways, night, fog, rain, and low light, with the remaining gaps reported honestly
Scalability	The pipeline should work for additional object classes and larger datasets by changing configuration, using SageMaker instance sizes
Usability	Results, reports, and the demo are understandable to non-specialists; commands and setup are documented
Maintainability	Modular code, clear branching, meaningful commits; documents and code kept on GitHub
Reliability	Training runs can resume from checkpoints; the system reports low-confidence results instead of hiding them

References

[1] Adam, M. A. A., and Tapamo, J. R. (2025). Enhancing YOLOv5 for Autonomous Driving: Efficient Attention-Based Object Detection on Edge Devices. Journal of Imaging, 11(8), 263. https://doi.org/10.3390/jimaging11080263

[2] Liang, S., Wu, H., Zhen, L., Hua, Q., Garg, S., Kaddoum, G., Hassan, M. M., and Yu, K. (2022). Edge YOLO: Real-Time Intelligent Object Detection System Based on Edge-Cloud Cooperation in Autonomous Vehicles. IEEE Transactions on Intelligent Transportation Systems, 23, 25345-25360. https://doi.org/10.1109/TITS.2022.3158253

[3] Alimov, M., and Meiramkhanov, T. (2024). Domain Generalization in Autonomous Driving: Evaluating YOLOv8s, RT-DETR, and YOLO-NAS with the ROAD-Almaty Dataset. arXiv:2412.12349. https://arxiv.org/abs/2412.12349

[4] Geiger, A., Lenz, P., Stiller, C., and Urtasun, R. (2013). Vision Meets Robotics: The KITTI Dataset. International Journal of Robotics Research, 32, 1231-1237. https://doi.org/10.1177/0278364913491297

[5] Yu, F., Chen, H., Wang, X., Xian, W., Chen, Y., Liu, F., Madhavan, V., and Darrell, T. (2020). BDD100K: A Diverse Driving Dataset for Heterogeneous Multitask Learning. CVPR, 2636-2645. https://arxiv.org/abs/1805.04687

[6] Jocher, G. (2020). Ultralytics YOLOv5, Version 7.0. https://github.com/ultralytics/yolov5

[7] Wang, Q., Wu, B., Zhu, P., Li, P., Zuo, W., and Hu, Q. (2019). ECA-Net: Efficient Channel Attention for Deep Convolutional Neural Networks. arXiv:1910.03151. https://arxiv.org/abs/1910.03151

[8] Hu, J., Shen, L., and Sun, G. (2017). Squeeze-and-Excitation Networks. arXiv:1709.01507. https://arxiv.org/abs/1709.01507